

1. Crear API Courses

Crear archivo `src/api/courses.ts`

```
import { http } from "../http";

export type DepartmentRef = {
  id: number;
  name: string;
};

export type Course = {
  id: number;
  code: string;
  title: string;
  departmentId: number;
};

export type CreateCourseDto = {
  code: string;
  title: string;
  departmentId: number;
};

export type UpdateCourseDto = Partial<CreateCourseDto>;

export const coursesApi = {
  list: () => http<Course[]>("/courses"),

  create: (dto: CreateCourseDto) =>
    http<Course>("/courses", {
      method: "POST",
      body: JSON.stringify(dto),
    }),
}
```

```

update: (id: number, dto: UpdateCourseDto) =>
  http<Course>(`/courses/${id}`, {
    method: "PATCH",
    body: JSON.stringify(dto),
  }),

remove: (id: number) =>
  http<void>(`/courses/${id}`, {
    method: "DELETE",
  }),
};

```

2. Crear TanStack Query Courses

Crear archivo `src/api/courses.queries.ts`

```

import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { coursesApi, type CreateCourseDto, type UpdateCourseDto } from "../courses";

const keys = {
  all: ["courses"] as const,
};

export function useCourses() {
  return useQuery({
    queryKey: keys.all,
    queryFn: coursesApi.list,
  });
}

```

```
export function useCreateCourse() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (dto: CreateCourseDto) => coursesApi.create(dto),
    onSuccess: () => qc.invalidateQueries({ queryKey: keys.all }),
  });
}

export function useUpdateCourse() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ id, dto }: { id: number; dto: UpdateCourseDto }) =>
      coursesApi.update(id, dto),
    onSuccess: () => qc.invalidateQueries({ queryKey: keys.all }),
  });
}

export function useDeleteCourse() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (id: number) => coursesApi.remove(id),
    onSuccess: () => qc.invalidateQueries({ queryKey: keys.all }),
  });
}
```

3. Componente CoursesPage (CRUD + Dropdown Departments)

Crear archivo `src/pages/CoursesPage.tsx`

```
import { useMemo, useState } from "react";
import {
  useCourses,
  useCreateCourse,
  useDeleteCourse,
  useUpdateCourse,
} from "../api/courses.queries";
import { useDepartment } from "../api/departaments.queries";

type EditingState = {
  id: number;
  code: string;
  title: string;
  departmentId: number | "";
};

export default function CoursesPage() {
  const { data: courses = [], isLoading, isError, error } = useCourses();
  const { data: departments = [] } = useDepartment();

  const createMut = useCreateCourse();
  const updateMut = useUpdateCourse();
  const deleteMut = useDeleteCourse();

  const [code, setCode] = useState("");
  const [title, setTitle] = useState("");
```

```

const [departmentId, setDepartmentId] = useState<number | "">("");

const [editing, setEditing] = useState<EditingState | null>(null);

const deptNameById = useMemo(() => {
  const map = new Map<number, string>();
  for (const d of departments as any[]) map.set(d.id, d.name);
  return map;
}, [departments]);

async function handleCreate(e: React.FormEvent) {
  e.preventDefault();

  // Validación mínima (además de required)
  if (!departmentId) return;

  await createMut.mutateAsync({
    code: code.trim(),
    title: title.trim(),
    departmentId: Number(departmentId),
  });

  setCode("");
  setTitle("");
  setDepartmentId("");
}

function startEdit(c: any) {
  setEditing({
    id: c.id,
    code: c.code,
  });
}

```

```

        title: c.title,
        departmentId: c.departmentId,
    });
}

function cancelEdit() {
    setEditing(null);
}

async function saveEdit() {
    if (!editing) return;
    if (!editing.departmentId) return;

    await updateMut.mutateAsync({
        id: editing.id,
        dto: {
            code: editing.code.trim(),
            title: editing.title.trim(),
            departmentId: Number(editing.departmentId),
        },
    });

    setEditing(null);
}

return (
    <div className="p-6 max-w-5xl mx-auto space-y-6">
        <div className="flex items-center justify-between">
            <h1 className="text-2xl font-bold">Courses</h1>
            <span className="text-sm text-slate-500">/courses</span>
        </div>

```

```
<form onSubmit={handleCreate} className="rounded-xl border bg-white p-4 space-y-3">
  <h2 className="font-semibold">Create Course</h2>

  <div className="grid gap-3 md:grid-cols-3">
    <div>
      <label className="block text-sm font-medium mb-1">Code</label>
      <input
        className="w-full rounded-lg border px-3 py-2"
        value={code}
        onChange={(e) => setCode(e.target.value)}
        placeholder="e.g., CS101"
        required
      />
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Title</label>
      <input
        className="w-full rounded-lg border px-3 py-2"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        placeholder="e.g., Introduction to Programming"
        required
      />
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Department</label>
      <select
        className="w-full rounded-lg border px-3 py-2"

```

```

        value={departmentId}
        onChange={(e) => setDepartmentId(Number(e.target.value))}
        required
      >
      <option value="">Select department</option>
      {(departments as any[]).map((d) => (
        <option key={d.id} value={d.id}>
          {d.name}
        </option>
      ))}
    </select>
  </div>
</div>

<button
  className="rounded-lg bg-black px-4 py-2 text-white disabled:opacity-50"
  disabled={createMut.isPending}
>
  {createMut.isPending ? "Creating..." : "Create"}
</button>

{createMut.isError && (
  <p className="text-sm text-red-600">Create error: {String(createMut.error)}</p>
)}
</form>

<div className="rounded-xl border bg-white overflow-hidden">
  <div className="p-4 border-b">
    {isLoading && <p className="text-sm text-slate-600">Loading courses...</p>}
    {isError && <p className="text-sm text-red-600">Error: {String(error)}</p>}
    {!isLoading && !isError && (

```

```

    <p className="text-sm text-slate-600">{courses.length} course(s)</p>
  )}
</div>

<div className="overflow-x-auto">
  <table className="w-full text-sm">
    <thead className="bg-slate-50 text-left">
      <tr>
        <th className="p-3">Code</th>
        <th className="p-3">Title</th>
        <th className="p-3">Department</th>
        <th className="p-3 w-64">Actions</th>
      </tr>
    </thead>

    <tbody>
      {(courses as any[]).map((c) => {
        const isEditing = editing?.id === c.id;

        const deptLabel =
          c.department?.name ??
          deptNameById.get(c.departmentId) ??
          `#${c.departmentId}`;

        return (
          <tr key={c.id} className="border-t">
            <td className="p-3">
              {isEditing ? (
                <input
                  className="w-full rounded border px-2 py-1"
                  value={editing!.code}

```

```

        onChange={(e) =>
            setEditing((prev) =>
                prev ? { ...prev, code: e.target.value } : prev
            )
        }
    />
) : (
    c.code
)}
</td>

<td className="p-3">
    {isEditing ? (
        <input
            className="w-full rounded border px-2 py-1"
            value={editing!.title}
            onChange={(e) =>
                setEditing((prev) =>
                    prev ? { ...prev, title: e.target.value } : prev
                )
            }
        />
    ) : (
        c.title
    )}
</td>

<td className="p-3">
    {isEditing ? (
        <select
            className="w-full rounded border px-2 py-1"

```

Number(e.target.value) }

```
value={editing!.departmentId}
onChange={(e) =>
  setEditing((prev) =>
    prev
    ? { ...prev, departmentId:
      : prev
    )
  }
>
  {(departments as any[]).map((d) => (
    <option key={d.id} value={d.id}>
      {d.name}
    </option>
  ))}
</select>
) : (
  deptLabel
)}
</td>

<td className="p-3">
  <div className="flex gap-2">
    {isEditing ? (
      <>
        <button
          className="rounded bg-black px-3 py-1 text-white
disabled:opacity-50"
          onClick={saveEdit}
          disabled={updateMut.isPending}
        >
```

```

        {updateMut.isPending ? "Saving..." : "Save"}
      </button>

      <button
        className="rounded border px-3 py-1"
        onClick={cancelEdit}
      >
        Cancel
      </button>
    </>
  ) : (
    <>
      <button
        className="rounded border px-3 py-1"
        onClick={() => startEdit(c)}
      >
        Edit
      </button>

      <button
        className="rounded bg-red-600 px-3 py-1 text-white"

        onClick={() => {
          if (!confirm("Delete this course?")) return;
          deleteMut.mutate(c.id);
        }}
        disabled={deleteMut.isPending}
      >
        Delete
      </button>
    </>
  )
}

```

disabled:opacity-50"

```

    })
    </div>
  </td>
</tr>
);
}}

{!isLoading && !isError && courses.length === 0 && (
  <tr>
    <td className="p-6 text-center text-slate-500" colSpan={4}>
      No courses found. Create the first one above.
    </td>
  </tr>
)}
</tbody>
</table>
</div>

{(updateMut.isError || deleteMut.isError) && (
  <div className="p-4 border-t">
    {updateMut.isError && (
      <p className="text-sm text-red-600">Update error: {String(updateMut.error)}</p>
    )}
    {deleteMut.isError && (
      <p className="text-sm text-red-600">Delete error: {String(deleteMut.error)}</p>
    )}
  </div>
)}
</div>
</div>
);

```


1. Instalación de dependencias

En la raíz del proyecto ejecutar el comando:

```
npm install chart.js react-chartjs-2
```

2. Crear archivo de datos simulados

Crea la carpeta `src/mocks/` y el archivo `src/mocks/dashboard.mock.ts`

```
export const dashboardMock = {
  completed: 120,
  pending: 45,
  cancelled: 10,

  monthlySales: [
    { month: "Enero", total: 1200 },
    { month: "Febrero", total: 1800 },
    { month: "Marzo", total: 900 },
    { month: "Abril", total: 2200 },
    { month: "Mayo", total: 3100 },
  ],

  topProducts: [
    { name: "Laptop", total: 35 },
    { name: "Mouse", total: 80 },
    { name: "Teclado", total: 40 },
    { name: "Monitor", total: 25 },
  ],
};
```

3. Simular la API

Crear archivo `src/api/dashboard.ts`:

```
import { dashboardMock } from "../mocks/dashboard.mock";

export async function getDashboardSummary() {

  // Simula un tiempo de carga como si consultara una API real
  await new Promise((resolve) =>
    setTimeout(resolve, 1000)
  );

  return dashboardMock;
}
```

4. Crear Query con TanStack Query

Crea el archivo `src/api/dashboard.queries.ts`

```
import { useQuery } from "@tanstack/react-query";
import { getDashboardSummary } from "../dashboard";

export function useDashboardSummary() {
  return useQuery({
    queryKey: ["dashboard"],
    queryFn: getDashboardSummary,
  });
}
```

5. Crear gráfico de barras

Crea carpeta `src/charts/` y el archivo `src/charts/SalesChart.tsx`

```
import { Chart as ChartJS, CategoryScale, LinearScale, BarElement, Tooltip, Legend } from "chart.js";

import { Bar } from "react-chartjs-2";

ChartJS.register(
  CategoryScale,
  LinearScale,
  BarElement,
  Tooltip,
  Legend
);

type Props = {
  data: {
    month: string;
    total: number;
  }[];
};

export default function SalesChart({ data }: Props) {

  const chartData = {
    labels: data.map((x) => x.month),
    datasets: [
      {
        label: "Ventas mensuales",
        data: data.map((x) => x.total),
        backgroundColor: [
          "rgba(59, 130, 246, 0.75)",
          "rgba(16, 185, 129, 0.75)",
          "rgba(245, 158, 11, 0.75)",
        ],
      },
    ],
  };

  return (
    <Bar
      data={chartData}
      options={{
        responsive: true,
        maintainAspectRatio: false,
      }}
    />
  );
}
```

```

        "rgba(239, 68, 68, 0.75)",
        "rgba(139, 92, 246, 0.75)",
    ],
    borderColor: [
        "rgb(37, 99, 235)",
        "rgb(5, 150, 105)",
        "rgb(217, 119, 6)",
        "rgb(220, 38, 38)",
        "rgb(124, 58, 237)",
    ],
    borderWidth: 2,
    borderRadius: 10,
    },
    ],
};

const options = {
    responsive: true,
    plugins: {
        legend: {
            position: "top" as const,
        },
    },
    scales: {
        y: {
            beginAtZero: true,
        },
    },
};

return <Bar data={chartData} options={options} />;
}

```

6. Crear gráfico tipo dona

Crear archivo `src/charts/ProductsChart.tsx`:

```
import {
  Chart as ChartJS,
  ArcElement,
  Tooltip,
  Legend,
} from "chart.js";

import { Doughnut } from "react-chartjs-2";

ChartJS.register(ArcElement, Tooltip, Legend);

type Props = {
  data: {
    name: string;
    total: number;
  }[];
};

export default function ProductsChart({ data }: Props) {

  const chartData = {
    labels: data.map((x) => x.name),
    datasets: [
      {
        label: "Productos",
        data: data.map((x) => x.total),
        backgroundColor: [
          "rgba(14, 165, 233, 0.8)",
          "rgba(34, 197, 94, 0.8)",
          "rgba(249, 115, 22, 0.8)",
          "rgba(168, 85, 247, 0.8)",
        ],
        borderColor: "#ffffff",
        borderWidth: 3,
      },
    ],
  };

  return <Doughnut data={chartData} />;
}
```

7. Crear Dashboard

Crear archivo `src/pages/DashboardPage.tsx`

```
import { useDashboardSummary } from "../../api/dashboard.queries";
import SalesChart from "../../charts/SalesChart";
import ProductsChart from "../../charts/ProductsChart";

export default function DashboardPage() {

  const { data, isLoading } =
    useDashboardSummary();

  if (isLoading) {
    return <p>Cargando dashboard...</p>;
  }

  return (
    <div className="space-y-6">

      <div>
        <h1 className="text-3xl font-bold text-slate-800">
          Dashboard
        </h1>
        <p className="text-slate-500">
          Indicadores simulados para análisis visual.
        </p>
      </div>

      <div className="grid md:grid-cols-3 gap-4">

        <div className="bg-blue-600 text-white rounded-xl p-4 shadow">
          <p>Completed</p>
          <h2 className="text-4xl font-bold">
            {data?.completed}
          </h2>
        </div>

        <div className="bg-amber-500 text-white rounded-xl p-4 shadow">
          <p>Pending</p>
          <h2 className="text-4xl font-bold">
            {data?.pending}
          </h2>
        </div>

        <div className="bg-rose-500 text-white rounded-xl p-4 shadow">
          <p>Cancelled</p>
          <h2 className="text-4xl font-bold">
            {data?.cancelled}
          </h2>
        </div>

      </div>

    </div>
  );
}
```

```

<div className="grid lg:grid-cols-2 gap-4">

  <div className="bg-white rounded-xl border p-4 shadow-sm">
    <h2 className="text-xl font-semibold mb-4">
      Ventas mensuales
    </h2>
    <SalesChart
      data={data?.monthlySales ?? []}
    />
  </div>

  <div className="bg-white rounded-xl border p-4 shadow-sm">
    <h2 className="text-xl font-semibold mb-4">
      Productos más vendidos
    </h2>
    <ProductsChart
      data={data?.topProducts ?? []}
    />
  </div>

</div>

</div>
);
}

```

1. Crear carpeta de Layout

Crear carpeta `src/layouts`

2. Crear Sidebar Layout Base

Crear Archivo `src/layouts/MainLayout.tsx`

```
import React from "react";

export default function MainLayout({
  sidebar,
  content,
}): {
  sidebar: React.ReactNode;
  content: React.ReactNode;
}) {
  return (
    <div className="min-h-screen flex bg-gray-100">
      {/* Sidebar */}
      <aside className="w-64 bg-white border-r p-4">
        {sidebar}
      </aside>
      {/* Main Content */}
      <main className="flex-1 p-6">
        {content}
      </main>
    </div>
  );
}
```

3. Crear Componente Sidebar

Crear carpeta `src/components`

Crear Archivo `src/components/SidebarMenu.tsx`

```
type Props = {
  current: string;
  onChange: (page: string) => void;
};

export default function SidebarMenu({ current, onChange }: Props) {
  return (
    <div className="space-y-4">
      <h2 className="text-xl font-bold">MiniPOS</h2>
      <nav className="flex flex-col gap-2">
        <button
          className={`text-left p-2 rounded ${current === "customers"
            ? "bg-black text-white" : "hover:bg-gray-100"}`}
          onClick={() => onChange("customers")}
        >
          Customers
        </button>
        <button
          className={`text-left p-2 rounded ${current === "departments"
            ? "bg-black text-white" : "hover:bg-gray-100"}`}
          onClick={() => onChange("departments")}
        >
          Departments
        </button>
      </nav>
    </div>
  );
}
```

4. Usar Layout en App

Editar el archivo `src/App.tsx`

```
import { useState } from 'react'
import './App.css'
import DepartamentsPage from "../pages/DepartamentPage";
import CustomersPage from '../pages/CustomersPage';
import MainLayout from "../layouts/MainLayout";
import SidebarMenu from "../components/SidebarMenu";

function App() {
  const [page, setPage] = useState("customers");

  function renderContent() {
    switch (page) {
      case "customers":
        return <CustomersPage />;
      case "departments":
        return <DepartamentsPage />;
      default:
        return <CustomersPage />;
    }
  }

  return (
    <MainLayout
      sidebar={<SidebarMenu current={page} onChange={setPage} />}
      content={renderContent()} />
  )
}

export default App
```

1. Capa API de autenticación

Crea el archivo `src/api/auth.ts`. Sigue el mismo patrón que `customers.ts` de la Guía 1:

```
import { http } from "../http";

export type AuthResponse = {
  token: string;
  username: string;
};

export type LoginDto = {
  username: string;
  password: string;
};

export type RegisterDto = {
  username: string;
  password: string;
};

export const authApi = {
  login: (dto: LoginDto) => http<AuthResponse>("/api/auth/login",
    { method: "POST", body: JSON.stringify(dto) }),
  register: (dto: RegisterDto) => http<AuthResponse>("/api/auth/register",
    { method: "POST", body: JSON.stringify(dto) }),
};
```

2. Contexto de autenticación (AuthContext)

Crea la carpeta src/context y el archivo src/context/AuthContext.tsx. Este contexto gestiona el estado global del usuario autenticado:

```
// src/context/AuthContext.tsx
import { createContext, useContext, useState, ReactNode } from "react";

type AuthUser = { token: string; username: string } | null;

type AuthContextType = {
  user: AuthUser;
  login: (token: string, username: string) => void;
  logout: () => void;
};

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  // Persistencia: carga token de localStorage al iniciar
  const [user, setUser] = useState<AuthUser>(() => {
    const token = localStorage.getItem("token");
    const username = localStorage.getItem("username");
    return token && username ? { token, username } : null;
  });

  function login(token: string, username: string) {
    localStorage.setItem("token", token);
    localStorage.setItem("username", username);
    setUser({ token, username });
  }

  function logout() {
    localStorage.removeItem("token");
    localStorage.removeItem("username");
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// Hook de acceso rápido
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be inside AuthProvider");
  return ctx;
}
```

3. Interceptor HTTP con el token

Modifica src/api/http.ts para que envíe automáticamente el token JWT en cada petición. Esto aplica a TODOS los endpoints:

```
// src/api/http.ts (versión actualizada)
export const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:8080";

export async function http<T>(path: string, options?: RequestInit): Promise<T> {
  // Lee el token guardado
  const token = localStorage.getItem("token");

  const res = await fetch(`${API_URL}${path}`, {
    headers: {
      "Content-Type": "application/json",
      // Agrega el header solo si existe token
      ...(token ? { "Authorization": `Bearer ${token}` } : {}),
      ...(options?.headers ?? {}),
    },
    ...options,
  });

  // Si el servidor responde 401, el token expiró → limpiar sesión
  if (res.status === 401) {
    localStorage.removeItem("token");
    localStorage.removeItem("username");
    window.location.href = "/login";
    throw new Error("Sesión expirada");
  }

  if (!res.ok) {
    const msg = await res.text();
    throw new Error(msg || `HTTP ${res.status}`);
  }

  if (res.status === 204) return undefined as T;
  return res.json() as Promise<T>;
}
```

4. Página de Login

Crea el archivo `src/pages/LoginPage.tsx`. Incluye validación de formulario en el cliente:

```
// src/pages/LoginPage.tsx
import { useState } from "react";
import { authApi } from "../api/auth";
import { useAuth } from "../context/AuthContext";

export default function LoginPage({ onSuccess }: { onSuccess: () => void }) {
  const { login } = useAuth();
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);

  // Validación del lado del cliente
  function validate(): string | null {
    if (!username.trim()) return "El usuario es obligatorio.";
    if (password.length < 6) return "La contraseña debe tener al menos 6 caracteres.";
    return null;
  }

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    const validationError = validate();
    if (validationError) { setError(validationError); return; }

    setLoading(true);
    setError(null);
    try {
      const res = await authApi.login({ username, password });
      login(res.token, res.username);
      onSuccess(); // navega a la app
    } catch (e) {
      setError("Credenciales inválidas. Intenta de nuevo.");
    } finally {
      setLoading(false);
    }
  }

  return (
    <div className="min-h-screen bg-slate-100 flex items-center justify-center">
      <div className="bg-white rounded-2xl shadow-md p-8 w-full max-w-sm">
        <h1 className="text-2xl font-bold text-slate-800 mb-6">Iniciar sesión</h1>

        {error && (
          <div className="mb-4 rounded-lg bg-red-50 border border-red-200 p-3 text-sm text-red-700">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4 noValidate">
          <div>
            <label className="block text-sm font-medium mb-1">Usuario</label>
            <input
              type="text"
              value={username}
              onChange={e => setUsername(e.target.value)}
              className="w-full rounded-lg border px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
              autoComplete="username"
            />
          </div>
          <div>
            <label className="block text-sm font-medium mb-1">Contraseña</label>
            <input
              type="password"
              value={password}
              onChange={e => setPassword(e.target.value)}
              className="w-full rounded-lg border px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500"
              autoComplete="current-password"
            />
          </div>
          <button
            type="submit"
            disabled={loading}
            className="w-full rounded-lg bg-blue-600 py-2 text-white font-medium hover:bg-blue-700 disabled:opacity-50"
          >
            {loading ? "Ingresando..." : "Ingresar"}
          </button>
        </form>
      </div>
    </div>
  );
}
```

5. Ruta protegida: PrivateRoute

Crea el componente `src/components/PrivateRoute.tsx`. Redirige al login si el usuario no está autenticado:

```
// src/components/PrivateRoute.tsx
import { useAuth } from "../context/AuthContext";

type Props = {
  children: React.ReactNode;
  fallback: React.ReactNode; // qué mostrar si no hay sesión
};

export default function PrivateRoute({ children, fallback }: Props) {
  const { user } = useAuth();
  return user ? <>{children}</> : <>{fallback}</>;
}
```

6. Actualizar main.tsx con el Provider

Envuelve la app con `AuthProvider` para que todos los componentes puedan acceder al contexto de autenticación. Edita `src/main.tsx`:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { AuthProvider } from "../context/AuthContext";
```

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
});
```

```
ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <AuthProvider>
      <QueryClientProvider client={queryClient}>
        <App />
      </QueryClientProvider>
    </AuthProvider>
  </React.StrictMode>,
);
```

7. Actualizar App.tsx con navegación por estado

Edita src/App.tsx para mostrar el LoginPage si no hay sesión activa, o la app completa si sí la hay:

```
import { useState } from 'react'
import './App.css'
//Contenido de la pagina
import CustomersPage from "../pages/CustomersPage";
import DepartamentPage from "../pages/DepartamentPage";
import TestMenuOptionPage from "../pages/TestMenuOptionPage";
//Organizador de la interfaz
import MainLayout from "../layouts/MainLayout";
//Contenedor del menú
import SidebarMenu from "../components/SidebarMenu";
import { useAuth } from "../context/AuthContext";
import LoginPage from "../pages/LoginPage";
import PrivateRoute from "../components/PrivateRoute";
```

```
function App() {

  const { user, logout } = useAuth();
  const [page, setPage] = useState("customers");
  const [menuOptions, setMenuOptions] = useState([
    {
      name: "customers",
      content: "Customers"
    },
    {
      name: "departments",
      content: "Departments"
    },
    {
      name: "tmo",
      content: "TMO"
    },
    {
      name: "about",
      content: "About..."
    },
    {
      name: "log-out",
      content: "Log out"
    }
  ]);
```

```
function renderContent() {
  switch (page) {
    case "customers":
      return <CustomersPage />;
    case "departments":
      return <DepartamentPage />;
    case "tmo":
      return <TestMenuOptionPage />;
    default:
      return <CustomersPage />;
  }
}
```

```
const sidebar = (
  <div>
    <SidebarMenu current={page} onChange={setPage} menuOptions={menuOptions}/>
    <div className="mt-6 border-t pt-4">
      <p className="text-xs text-gray-500 mb-2">Hola, {user?.username}</p>
      <button
        onClick={logout}
        className="text-sm text-red-600 hover:underline"
      >
        Cerrar sesión
      </button>
    </div>
  </div>
);
```

```
return (
  <PrivateRoute fallback={<LoginPage onSuccess={() => { }} />}>
    <MainLayout sidebar={sidebar} content={renderContent()} />
  </PrivateRoute>
);
```

```
}
```

```
export default App
```

1. Instalar

```
npm install @tanstack/react-query
```

2. Crear QueryClient y provider (montaje global)

En el archivo `src/main.tsx`:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
});

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </React.StrictMode>,
);
```

Elementos importantes

- `QueryClient`: "cerebro" del cache
- `QueryClientProvider`: hace disponible `QueryClient` en toda la app
- `retry: 1`: intenta 1 vez extra si falla
- `refetchOnWindowFocus: false`: evita refrescos inesperados al cambiar de ventana

3. Crear “keys”

TanStack Query guarda información por llave. Si cambian datos, invalida esa llave.

Ejemplo:

- Llave del listado customers: ["customers"]

4. Crear hooks de query y mutation

Crear archivo `src/api/customers.queries.ts`:

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { customersApi, type CreateCustomerDto, type UpdateCustomerDto } from "../customers";

const keys = {
  all: ["customers"] as const,
};

export function useCustomers() {
  return useQuery({
    queryKey: keys.all,
    queryFn: customersApi.list,
  });
}

export function useCreateCustomer() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (dto: CreateCustomerDto) => customersApi.create(dto),
    onSuccess: () => qc.invalidateQueries({ queryKey: keys.all }),
  });
}

export function useUpdateCustomer() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ id, dto }: { id: number; dto: UpdateCustomerDto }) =>
      customersApi.update(id, dto),
    onSuccess: () => qc.invalidateQueries({ queryKey: keys.all }),
  });
}

export function useDeleteCustomer() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (id: number) => customersApi.remove(id),
    onSuccess: () => qc.invalidateQueries({ queryKey: keys.all }),
  });
}
```

Elementos importantes

- `useQuery`: lectura (GET)
- `useMutation`: escritura (POST/PATCH/DELETE)
- `invalidateQueries`: “tu cache está viejo, vuelve a pedir al servidor”

5. UI con create + list + delete

Actualiza `CustomersPage.tsx` para usar TanStack Query:

- No trabaja con `useEffect` ni loading manual
- `useCustomers()` implementa `isLoading`, `isError`, `data`
- `create/delete` son `mutate` o `mutateAsync`

```
import { useState } from "react";
import { useCreateCustomer, useCustomers, useDeleteCustomer } from "../api/customers.queries";

export default function CustomersPage() {
  const { data = [], isLoading, isError, error, refetch } = useCustomers();
  const createMut = useCreateCustomer();
  const deleteMut = useDeleteCustomer();

  const [fullName, setFullName] = useState("");
  const [email, setEmail] = useState("");

  async function onCreate(e: React.FormEvent) {
    e.preventDefault();
    await createMut.mutateAsync({ fullName, email });
    setFullName("");
    setEmail("");
  }

  return (
    <div className="min-h-screen bg-slate-50">
      <header className="border-b bg-white">
        <div className="mx-auto max-w-5xl px-4 py-4 flex items-center justify-between">
          <h1 className="text-xl font-semibold">Customers</h1>
          <button className="rounded-lg border px-3 py-2" onClick={() => refetch()}>
            Reintentar/Refrescar
          </button>
        </div>
      </header>

      <main className="mx-auto max-w-5xl px-4 py-6 space-y-4">
        <form onSubmit={onCreate} className="rounded-xl border bg-white p-4 space-y-3">
          <p className="text-sm text-slate-600">
            <b>Mutation (POST)</b>: crea customer y luego invalida cache para refrescar listado.
          </p>
        </form>
      </main>
    </div>
  );
}
```

```

<div className="grid gap-3 md:grid-cols-2">
  <div>
    <label className="block text-sm font-medium mb-1">Nombre</label>
    <input
      className="w-full rounded-lg border px-3 py-2"
      value={fullName}
      onChange={(e) => setFullName(e.target.value)}
      required
    />
  </div>

  <div>
    <label className="block text-sm font-medium mb-1">Email</label>
    <input
      className="w-full rounded-lg border px-3 py-2"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      required
    />
  </div>
</div>

<button
  className="rounded-lg bg-black px-4 py-2 text-white disabled:opacity-50"
  disabled={createMut.isPending}
>
  {createMut.isPending ? "Creando..." : "Crear"}
</button>

{createMut.isError && (
  <p className="text-sm text-red-600">Error creando: {String(createMut.error)}</p>
)}
</form>

<div className="rounded-xl border bg-white">
  <div className="p-4 border-b">
    {isLoading && <p className="text-sm text-slate-600">Cargando...</p>}
    {isError && <p className="text-sm text-red-600">Error: {String(error)}</p>}
    {!isLoading && !isError && (
      <p className="text-sm text-slate-600">{data.length} registro(s)</p>
    )}
  </div>

  <div className="overflow-x-auto">
    <table className="w-full text-sm">
      <thead className="bg-slate-50 text-left">

```

```

        <tr>
          <th className="p-3">Nombre</th>
          <th className="p-3">Email</th>
          <th className="p-3 w-28">Acción</th>
        </tr>
      </thead>
      <tbody>
        {data.map((c) => (
          <tr key={c.id} className="border-t">
            <td className="p-3">{c.fullName}</td>
            <td className="p-3">{c.email}</td>
            <td className="p-3">
              <button
                className="rounded-md border px-2 py-1 hover:bg-slate-50 disabled:opacity-50"
                disabled={deleteMut.isPending}
                onClick={() => {
                  if (!confirm("¿Seguro que deseas borrar este customer?")) return;
                  deleteMut.mutate(c.id);
                }}
              >
                Borrar
              </button>
            </td>
          </tr>
        ))}
        {!isLoading && !isError && data.length === 0 && (
          <tr>
            <td className="p-6 text-center text-slate-500" colspan={3}>
              No hay registros.
            </td>
          </tr>
        )}
      </tbody>
    </table>
  </div>

  {deleteMut.isError && (
    <div className="p-4">
      <p className="text-sm text-red-600">Error borrando: {String(deleteMut.error)}</p>
    </div>
  )}
</div>
</main>
</div>
);
}

```

1. Crear el proyecto con Vite (React + TypeScript)

Uso de Vite

- Es rápido (hot reload inmediato)
- Genera estructura moderna de React
- Evita configuraciones manuales complejas

Ejecutar:

```
npm create vite@latest minipos-web -- --template react-ts
cd minipos-web
npm install
npm run dev
```

Archivos importantes:

- `src/main.tsx`: punto de entrada donde React “monta” la app
- `src/App.tsx`: componente principal
- `tsconfig.json`: configuración TypeScript (compilación, tipos)
- `vite.config.ts`: configuración Vite

2. Conectar el FrontEnd con tu backend

Crear `.env` en la raíz del frontend

```
VITE_API_URL=http://localhost:3000
```

3. Instalar y configurar Tailwind CSS

Es un framework de CSS “utility-first”: en vez de escribir hojas de estilo grandes, compone estilos directamente con clases.

Ejemplo:

- `bg-white` = fondo blanco
- `rounded-xl` = bordes redondeados
- `p-4` = padding
- `text-slate-600` = color de texto

Instalación

```
npm install -D tailwindcss postcss autoprefixer
npm install @tailwindcss/postcss postcss autoprefixer -D
```

En la raíz del proyecto crear archivo `tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*..{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

En la raíz del proyecto crear archivo `postcss.config.js`

```
export default {
  plugins: {
    "@tailwindcss/postcss": {},
    autoprefixer: {},
  },
}
```

Editar el archivo `src/index.css`

```
@import "tailwindcss";
```

Editar el archivo `src/App.tsx`

```
<div className="min-h-screen bg-slate-900 flex items-center justify-center">
  <h1 className="text-4xl font-bold text-sky-400 drop-shadow-lg">
    ¡Tailwind funcionando!
  </h1>
</div>
```

4. Crear un “cliente HTTP” genérico

Crear carpeta y archivo `src/api/http.ts`

```
export const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:3000";

export async function http<T>(path: string, options?: RequestInit): Promise<T> {
  const res = await fetch(`${API_URL}${path}`, {
    headers: { "Content-Type": "application/json", ...(options?.headers ?? {}) },
    ...options,
  });

  if (!res.ok) {
    const msg = await res.text();
    throw new Error(msg || `HTTP ${res.status}`);
  }

  if (res.status === 204) return undefined as T;

  return res.json() as Promise<T>;
}
```

Contenido:

- `API_URL`: URL base del backend (env o fallback)
- `http<T>`: función genérica que devuelve un tipo T (TypeScript)
- `res.ok`: si status no está entre 200–299, lanzamos error
- `204`: no hay contenido, evitamos `res.json()` (si no, rompe)
- `return res.json()`: convertimos respuesta a JSON

5. Crear la “capa API”

Separación del “API” y el “UI”

- La UI no debe saber cómo se llama el endpoint o cómo se hace el fetch
- Si cambia el backend, solo cambia api/

Crea archivo `src/api/customers.ts`:

```
import { http } from "../http";

export type Customer = {
  id: number;
  fullName: string;
  email: string;
  phone?: string | null;
  createdAt?: string;
};

export type CreateCustomerDto = {
  fullName: string;
  email: string;
  phone?: string;
};

export type UpdateCustomerDto = Partial<CreateCustomerDto>;

export const customersApi = {
  list: () => http<Customer[]>("/customers"),
  create: (dto: CreateCustomerDto) =>
    http<Customer>("/customers", { method: "POST", body: JSON.stringify(dto) }),
  update: (id: number, dto: UpdateCustomerDto) =>
    http<Customer>(`/customers/${id}`, { method: "PATCH", body: JSON.stringify(dto) }),
  remove: (id: number) => http<void>(`/customers/${id}`, { method: "DELETE" }),
};
```

6. Hooks avanzados: Debounce + Deferred

Crear carpetas y archivo `src/hooks/useDebouncedValue.ts`:

```
import { useEffect, useState } from "react";

export function useDebouncedValue<T>(value: T, delayMs = 300) {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);

  return debounced;
}
```

Elementos importantes:

- `useEffect`: corre cuando cambia value
- `setTimeout`: espera delayMs
- `clearTimeout`: cancela el anterior si el usuario siguió escribiendo
- Resultado: solo filtra cuando el usuario "hace una pausa"
- `useDeferredValue`: Le dice a React: "esta actualización no es urgente". Ayuda cuando el render es pesado.

7. Página de Customers

Crear carpetas y archivo `src/pages/CustomersPage.tsx`

```
import { useEffect, useMemo, useState, useDeferredValue } from "react";
import { customersApi, type Customer } from "../api/customers";
import { useDebounceValue } from "../hooks/useDebounceValue";

export default function CustomersPage() {
  const [rows, setRows] = useState<Customer[]>([]);
  const [loading, setLoading] = useState(false);
  const [err, setErr] = useState<string | null>(null);

  const [q, setQ] = useState("");
  const debouncedQ = useDebounceValue(q, 250);
  const deferredQ = useDeferredValue(debouncedQ);

  // useEffect: cargar datos al entrar
  useEffect(() => {
    (async () => {
      setLoading(true);
      setErr(null);
      try {
        const data = await customersApi.list();
        setRows(data);
      } catch (e) {
        setErr(String(e));
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  // useMemo: evita recalcular filtrado si no cambia q o rows
  const filtered = useMemo(() => {
    const term = deferredQ.trim().toLowerCase();
    if (!term) return rows;
    return rows.filter((c) =>
      `${c.fullName} ${c.email} ${c.phone ?? ""}`.toLowerCase().includes(term),
    );
  }, [rows, deferredQ]);

  return (
    <div className="min-h-screen bg-slate-50">
      <header className="border-b bg-white">
        <div className="mx-auto max-w-5xl px-4 py-4 flex items-center justify-between">
          <h1 className="text-xl font-semibold">MiniPOS – Customers</h1>
          <span className="text-sm text-slate-500">Backend: /customers</span>
        </div>
      </header>

      <main className="mx-auto max-w-5xl px-4 py-6 space-y-4">
        <div className="rounded-xl border bg-white p-4">
          <label className="block text-sm font-medium mb-2">Buscar</label>
          <input
            value={q}
            onChange={(e) => setQ(e.target.value)}
            placeholder="Nombre, email o teléfono..."
            className="w-full rounded-lg border bg-white px-3 py-2"
          />
          <p className="mt-2 text-xs text-slate-500">
            Debounce + Deferred: reduce recalculos y mejora fluidez.
          </p>
        </div>

        <div className="rounded-xl border bg-white">
```

```

<div className="p-4 border-b">
  {loading && <p className="text-sm text-slate-600">Cargando...</p>}
  {err && <p className="text-sm text-red-600">Error: {err}</p>}
  {!loading && !err && (
    <p className="text-sm text-slate-600">{filtered.length} registro(s)</p>
  )}
</div>

<div className="overflow-x-auto">
  <table className="w-full text-sm">
    <thead className="bg-slate-50 text-left">
      <tr>
        <th className="p-3">Nombre</th>
        <th className="p-3">Email</th>
        <th className="p-3">Teléfono</th>
      </tr>
    </thead>
    <tbody>
      {filtered.map((c) => (
        <tr key={c.id} className="border-t">
          <td className="p-3">{c.fullName}</td>
          <td className="p-3">{c.email}</td>
          <td className="p-3">{c.phone ?? "-"}</td>
        </tr>
      ))}
      {!loading && !err && filtered.length === 0 && (
        <tr>
          <td className="p-6 text-center text-slate-500" colspan={3}>
            No hay datos. Crea algunos con Postman (POST
/customers).
          </td>
        </tr>
      )}
    </tbody>
  </table>
</div>
</div>
</main>
</div>
);
}

```

Elementos importantes:

- Ciclo de vida: useEffect para carga inicial
- Rendimiento: useMemo, useDebounceValue, useDeferredValue
- UX visual: layout, tabla, estados (loading/error/empty)

8. Integrar la página en App.tsx

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'
import CustomersPage from './pages/CustomersPage';

function App() {
  const [count, setCount] = useState(0)

  return (<CustomersPage />)
}

export default App
```